

```

const express = require("express");
const Anthropic = require("@anthropic-ai/sdk");
const cors = require("cors");
const rateLimit = require("express-rate-limit");

const app = express();
const anthropic = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });

app.use(cors());
app.use(express.json({ limit: "50mb" }));

// Rate limit: max 10 solve requests per IP per hour
const solveLimiter = rateLimit({
  windowMs: 60 * 60 * 1000,
  max: 10,
  message: { error: "Too many requests, please try again later." },
});

// In-memory usage store (replace with Supabase later)
const usageStore = {};

function getUsage(userId) {
  if (!usageStore[userId]) usageStore[userId] = { uses: 0, subscribed: false };
  return usageStore[userId];
}

// — Health check —————
app.get("/", (req, res) => {
  res.json({ status: "Test Taker API running  " });
});

// — Check usage —————
app.get("/api/usage/:userId", (req, res) => {
  const usage = getUsage(req.params.userId);
  res.json({
    uses: usage.uses,
    freeUsesLeft: Math.max(0, 3 - usage.uses),
    subscribed: usage.subscribed,
    canSolve: usage.subscribed || usage.uses < 3,
  });
});

// — Solve test —————
app.post("/api/solve", solveLimiter, async (req, res) => {

```

```

try {
  const { userId, subject, text, images } = req.body;

  if (!userId) return res.status(400).json({ error: "userId required" });
  if (!text && (!images || images.length === 0)) {
    return res.status(400).json({ error: "No test content provided" });
  }

  // Check usage
  const usage = getUsage(userId);
  if (!usage.subscribed && usage.uses >= 3) {
    return res.status(402).json({
      error: "upgrade_required",
      message: "You've used your 3 free solves. Upgrade to Test Taker Pro for $"
    });
  }

  // Build content for Claude
  const contentParts = [];

  // Add images if provided (base64)
  if (images && images.length > 0) {
    for (const img of images) {
      contentParts.push({
        type: "image",
        source: {
          type: "base64",
          media_type: img.mimeType || "image/jpeg",
          data: img.data,
        },
      });
    }
  }

  // Add extracted text if provided
  if (text) {
    contentParts.push({
      type: "text",
      text: `=== TEST CONTENT ===\n${text}\n=== END TEST ===`,
    });
  }

  // The magic prompt - maps passages/images to their questions
  contentParts.push({
    type: "text",
    text: `You are an expert test solver. Analyze this ${subject || "general"}

```

IMPORTANT – Follow these steps:

1. First scan the ENTIRE test to identify:

- Reading passages (text blocks that questions refer to)
- Diagrams or images tied to specific questions
- Question groups that share context
- Standalone questions

2. For EACH question, identify which passage/image/context it belongs to

3. Answer EVERY question using only its associated context

4. Format your response EXACTLY like this, one line per question:

Q1: C

Q2: B - Photosynthesis converts sunlight to energy

Q3: 1865

Q4: A

Rules:

- Use "Q<number>: <answer>" format strictly
- For multiple choice include the letter first, then brief explanation if helpful
- Do not skip any questions
- Do not add intro text or summaries
- If a question references a passage or image, make sure you are using the RIGHT

Answer every single question now:`,

```
});
```

```
// Call Claude
```

```
const message = await anthropic.messages.create({  
  model: "claude-sonnet-4-20250514",  
  max_tokens: 8000,  
  messages: [{ role: "user", content: contentParts }],  
});
```

```
const responseText = message.content.map((b) => b.text || "").join("").trim()
```

```
// Parse Q<n>: <answer> format
```

```
const lines = responseText.split("\n");
```

```
const answers = [];
```

```
for (const line of lines) {
```

```
  const m = line.trim().match(/^Q(\d+):\s*(.+)/i);
```

```
  if (m) {
```

```
    answers.push({
```

```
      q: parseInt(m[1]),
```

```
      a: m[2].trim(),
```

```
      conf: 95,
```

```
    });
```

```

    }
  }

  // Fallback: numbered lines
  if (answers.length === 0) {
    for (const line of lines) {
      const m = line.trim().match(/^(\\d+)(\\.):\\s+(.+)/);
      if (m) answers.push({ q: parseInt(m[1]), a: m[2].trim(), conf: 88 });
    }
  }

  if (answers.length === 0) {
    return res.status(500).json({
      error: "Could not parse answers",
      raw: responseText.substring(0, 500),
    });
  }

  // Increment usage
  if (!usage.subscribed) {
    usageStore[userId].uses += 1;
  }

  res.json({
    answers,
    usesLeft: Math.max(0, 3 - usageStore[userId].uses),
    subscribed: usage.subscribed,
  });
} catch (err) {
  console.error("Solve error:", err);
  res.status(500).json({ error: err.message || "Server error" });
}
});

// — Mark as subscribed (hook this up to RevenueCat webhook later) —
app.post("/api/subscribe", (req, res) => {
  const { userId } = req.body;
  if (!userId) return res.status(400).json({ error: "userId required" });
  usageStore[userId] = { ...getUsage(userId), subscribed: true };
  res.json({ success: true, subscribed: true });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Test Taker API running on port ${PORT}`));

```